

为什么 Transformer 最终压过 RNN

长期依赖 · 信息路径长度 · 训练并行性 · Self-Attention 的代价

本讲定位：这是“模型架构演化”阶段的关键过渡课。我们不把结论简化成“Transformer 一定比 RNN 好”，而是具体回答：在大规模序列建模和 LLM 预训练中，Transformer 为什么更容易扩展；它又为这种能力支付了哪些计算与显存成本。

学完后你应该能够：

- 从依赖图角度解释 RNN、LSTM/GRU 与 Self-Attention 的差异。
- 区分“总计算量”和“串行依赖深度”这两个不同概念。
- 解释为什么 Causal Transformer 在训练时仍能并行计算所有位置。
- 解释为什么自回归生成阶段依旧必须逐 Token。
- 看懂标准 Self-Attention 的 T^2 代价，并理解长上下文为何困难。
- 把这些基础重新接回 GPT / LLM / Agent Systems。

1. 先回顾：RNN 真正的结构约束是什么？

上一讲已经看到，普通 RNN、LSTM 和 GRU 虽然内部公式不同，但时间轴都保留同一个基本递推形式：

$$\text{state}_t = f(x_t, \text{state}_{t-1})$$

这句话的含义不是“RNN 比较老”，而是：第 t 个时间步的状态必须等第 $t-1$ 个状态算出来以后才能确定。这是一种数据依赖（data dependency），不是单纯换一块更快 GPU 就能消掉的依赖。

例如长度为 5 的序列：

$$x_1 \rightarrow h_1 \rightarrow h_2 \rightarrow h_3 \rightarrow h_4 \rightarrow h_5$$

如果我们想让第 5 个位置利用第 1 个位置的信息，这个影响需要穿过 $h_1 \rightarrow h_2 \rightarrow h_3 \rightarrow h_4 \rightarrow h_5$ 的递推链。LSTM 的门控可以改善这条链上的信息与梯度保存，但结构上的时间路径仍然存在。

这会带来两个直接后果：

- 长程依赖的信息/梯度路径随距离增加。
- 训练时，同一层的时间步骤很难全部同时算完。

这两点正是 Transformer 设计所改变的核心。

2. Self-Attention 改变了什么: Token 不再只通过相邻状态传话

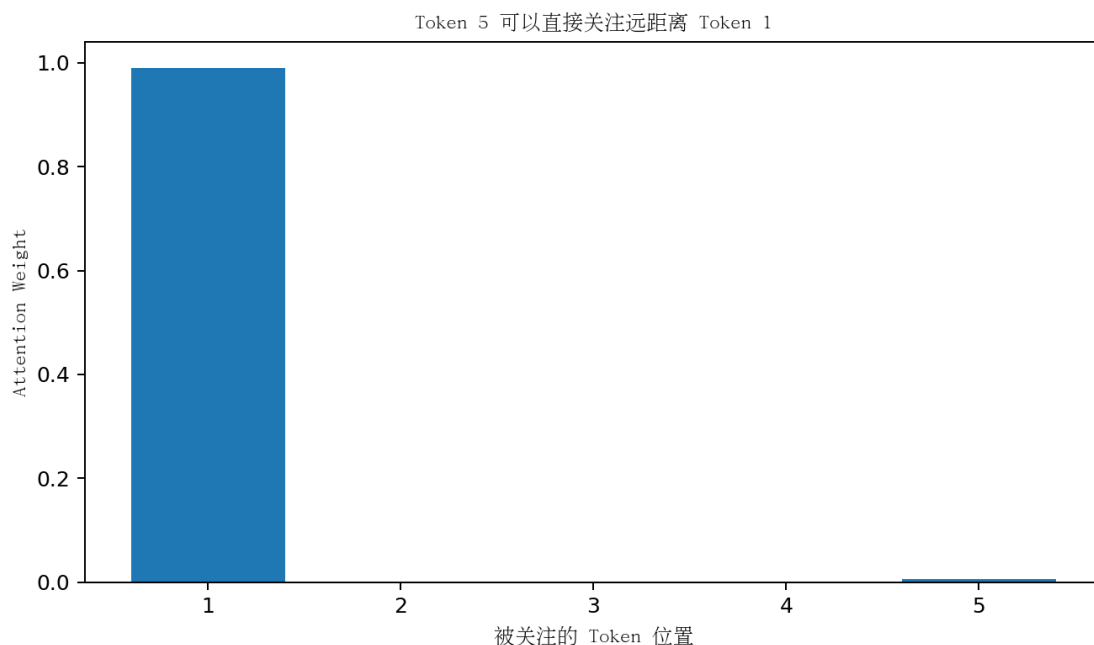
Self-Attention 的关键不是“有 Q/K/V 这些名字”，而是它允许每个 Query 直接与一组 Key 计算匹配分数，然后直接聚合相应 Value。

$$\text{Attention}(Q,K,V) = \text{softmax}(QK^T / \sqrt{d_k}) V$$

假设现在处理 Token 128。在 Causal Self-Attention 中, Token 128 虽然不能看 129 以后的位置, 但它可以在同一层里直接给 Token 1 很高的 Attention Weight。也就是说:

Token 128 → 直接读取 Token 1 的 Value

这与 RNN 的“Token 1 的信息不断经过 127 个递推状态传到 Token 128”是结构上完全不同的路径。

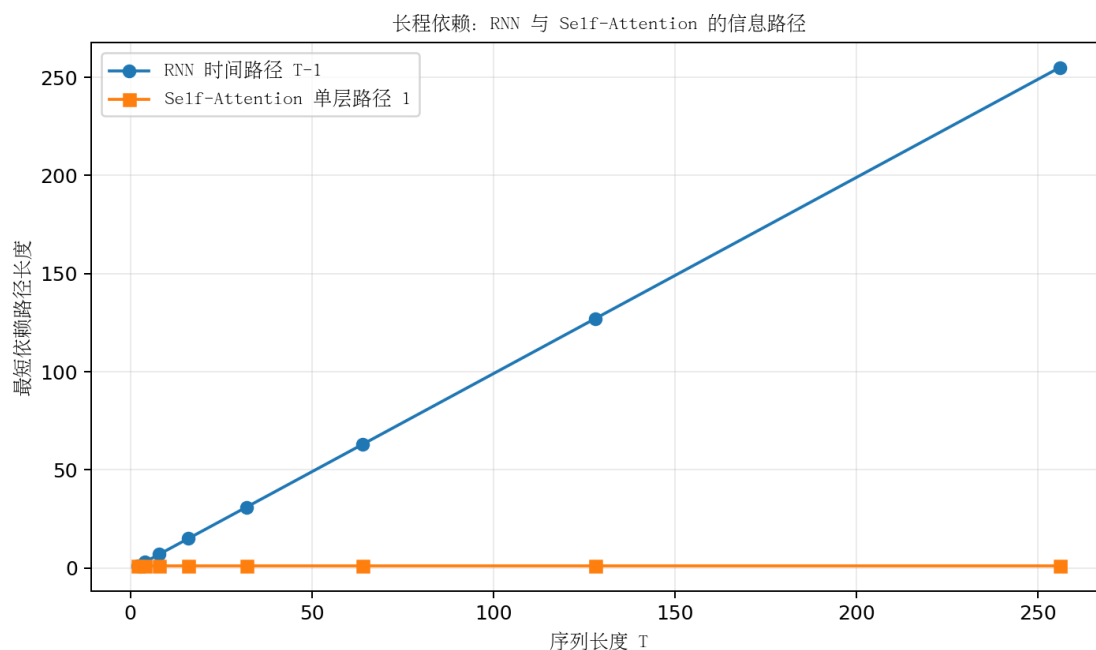


实验中, 我们故意让 Token 5 对 Token 1 的 score 最高。Softmax 后, Token 1 获得约 0.9906 的权重, 所以远距离信息可以在一个 Attention Layer 中直接进入 Token 5 的新表示。

3. “路径长度”为什么重要？

这里的路径长度，不是现实中的 Token 距离，而是：一个位置的信息要影响另一个位置，需要跨过多少个计算依赖节点。

结构	Token i 影响 Token j 的典型最短路径	随着距离 $j-i$ 增大
单向 RNN / LSTM / GRU	约 $j-i$ 个时间递推	线性增长
单层全连接 Self-Attention	一次直接 Attention 交互	基本不随 Token 距离增长



当 $T=128$ 时，最简 RNN 中 Token 1 $\rightarrow$ Token 128 的时间路径约为 127，而单层 Self-Attention 可以直接建立联系。

需要注意：路径更短不等于模型自动学会所有长程规律。Attention 仍然需要通过训练学到“应该关注谁”。这里只是在说，架构允许建立短路径，而不是保证一定利用好。

4. 为什么短路径也会影响 Gradient?

Backpropagation 仍然遵循你已经学过的 Chain Rule。RNN 的长程影响需要经过很多时间步骤，因此会出现很多 Jacobian / 局部导数的连乘。最小化地写，可以想成：

$$d h_T / d h_1 \approx J_T \cdot J_{\{T-1\}} \cdot \dots \cdot J_2$$

如果这些局部因子的典型尺度小于 1，连续相乘容易衰减；大于 1，则容易爆炸。LSTM/GRU 用门控改善了这个问题，但没有删除整条时间递推。

Self-Attention 中，远处 Token 的 Value 可以通过一个 Attention 层直接进入当前表示，因此架构上存在更短的信息与梯度路径。这也是 Transformer 更容易学习长程依赖的原因之一。

但不要把结论说成“Transformer 没有梯度问题”。Transformer 仍然会有深度方向的 Gradient Flow、Residual、Normalization、Initialization 等问题。它主要缩短的是序列位置之间的交互路径。

你以前在 LLM 课程里见过的 Residual + LayerNorm / RMSNorm，现在可以重新理解：它们主要帮助深层 Block 方向的稳定；Self-Attention 则改变 Token 与 Token 之间的信息路由。

5. 第二个关键优势：训练时的时间轴并行性

RNN 在同一层上存在：

$$h_t \text{ 依赖 } h_{t-1}$$

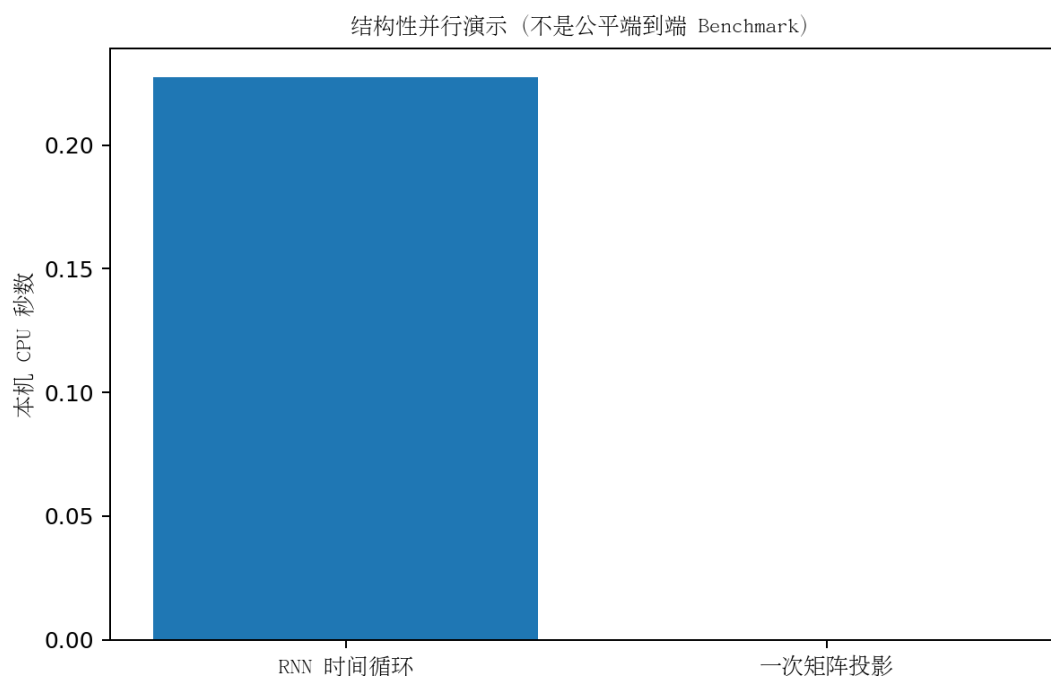
所以即使每一步内部是矩阵运算，时间步之间仍有“先算前一个，再算下一个”的依赖。

而 Transformer 一层中的 Q/K/V Projection：

$$Q = XW_Q, K = XW_K, V = XW_V$$

可以对所有 Token 的 X 一次做大矩阵乘法。随后 QK^T 也可以一次得到所有 Query-Key Pair 的 score。GPU / TPU 对这种大规模矩阵运算极其擅长。

这并不一定意味着 Transformer 的总 FLOPs 更少。事实上标准 Self-Attention 的 score 计算会有 T^2 项。但它把大量工作改造成更容易并行的矩阵计算，这对大规模预训练非常关键。



本机 CPU 的粗略演示中，Python RNN 时间循环远慢于一次矩阵投影。但这不是公平端到端性能基准，只能说明“依赖结构允许的并行形式不同”。

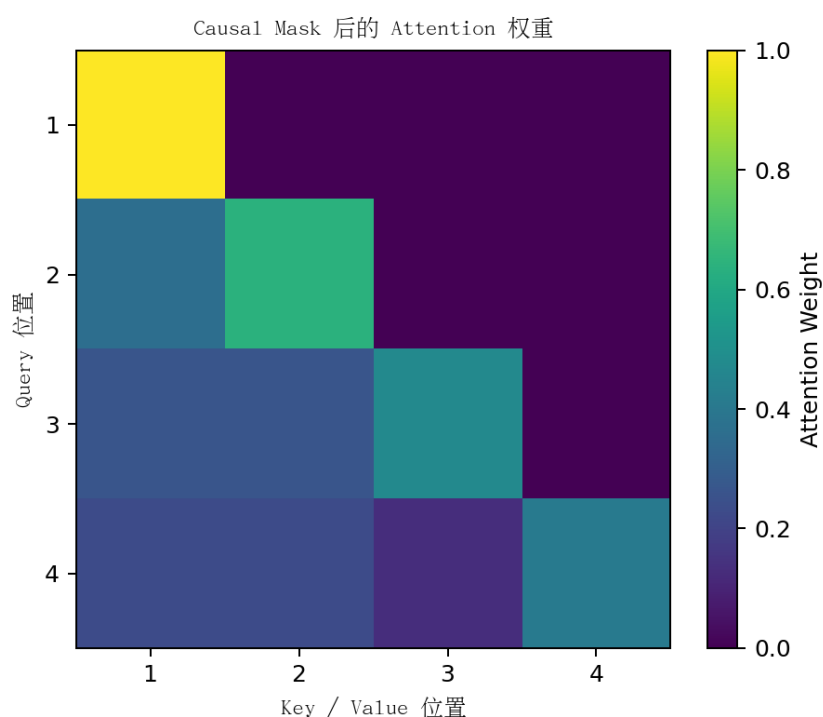
6. Causal Mask: 并行训练为什么没有偷看未来?

GPT 类模型有一个额外约束: 位置 t 预测下一个 Token 时, 不能读取未来位置。乍看之下似乎又必须像 RNN 那样一步一步算。关键在于: 训练时完整目标序列已经知道, 我们可以一次构造整个 $T \times T$ score matrix, 然后把未来位置 Mask 掉。

$$\text{scores} = QK^T / \sqrt{d_k}$$

$$\text{scores}[i,j] = -\infty \text{ if } j > i$$

之后一次对所有行做 Softmax。未来位置因为是 $-\infty$, Softmax 权重变成 0。这样每个位置都遵守因果限制, 但所有训练位置仍然可以在矩阵层面一起计算。



图中上三角未来位置权重全部为 0, 但 4 个 Query 行是在同一次矩阵计算中得到的。

所以要严格区分: 因果依赖不等于训练计算必须逐位置执行。Mask 能把约束编码进矩阵运算。

7. 但为什么自回归生成仍然是串行的？

训练和生成的条件不一样。训练时，目标序列已经在数据中存在，所以所有位置的输入 Token 都已知；生成时，第 $t+1$ 个新 Token 本身还不存在。

$$x_1 \dots x_t \rightarrow \text{model} \rightarrow \text{sample } x_{\{t+1\}}$$

$$x_1 \dots x_t, x_{\{t+1\}} \rightarrow \text{model} \rightarrow \text{sample } x_{\{t+2\}}$$

因此，第 2 个新 Token 依赖第 1 个新 Token 的实际采样结果，第 3 个又依赖前两个。这个自回归时间轴无法像训练那样完全并行。

这也是为什么你以前在 LLM 课程里学过 KV Cache：生成阶段我们不希望每次把全部历史 K/V 重算，于是缓存过去 Token 的 K/V，只计算新 Token 的部分。

所以一个更准确的说法是：

Transformer 极大改善了训练阶段和层内 Token 交互的并行性；但 GPT 式自回归解码仍然存在逐 Token 串行瓶颈。

Agent Systems 中如果 Agent 连续“思考 → 调工具 → 观察结果 → 再思考”，外部环境反馈也形成新的串行链，这不是 Self-Attention 能直接消掉的。

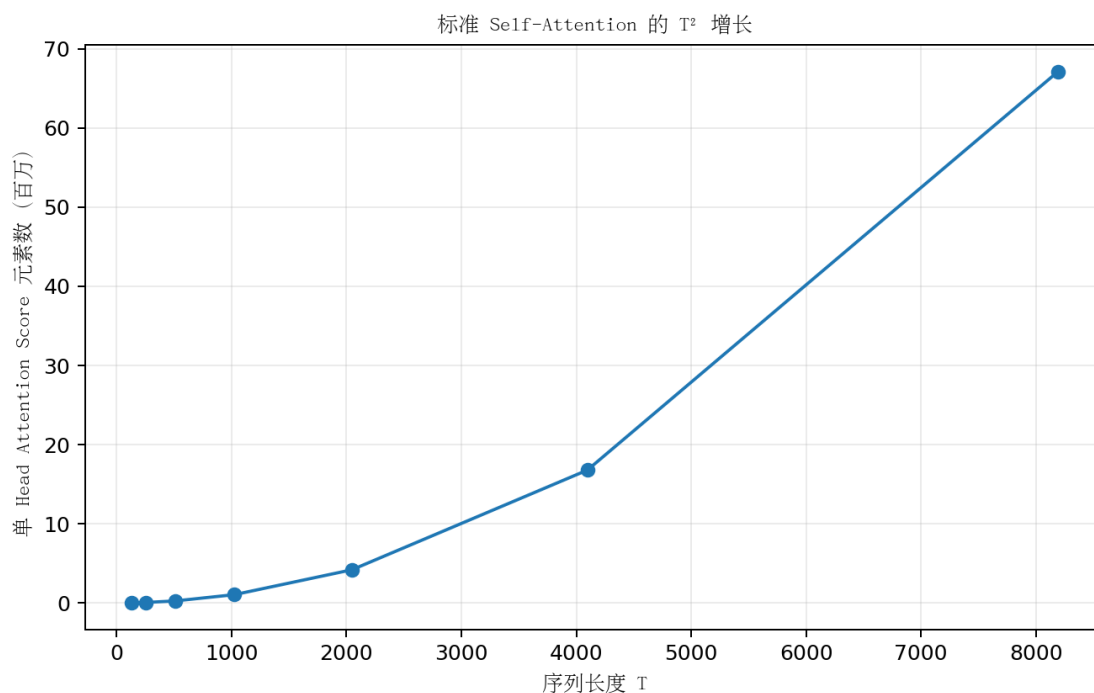
8. Transformer 的代价：标准 Self-Attention 为什么是 T^2 ？

如果序列有 T 个 Token，每一个 Query 都与 T 个 Key 计算 score，那么 score matrix 的 Shape 是：

$$(T, d_k) @ (d_k, T) \rightarrow (T, T)$$

因此 score 数量为：

$$T \times T = T^2$$



例如单个 Head，仅仅保存原始 fp16 score matrix: $T=4096$ 时约 32 MiB, $T=8192$ 时约 128 MiB。真实训练还要考虑多个 Head、多个 Layer、Softmax 中间量、Backward 保存的激活等，因此实际显存压力更大。

这就是为什么“Transformer 更容易并行”与“长上下文很贵”可以同时成立：它把时间串行依赖换成了大规模 all-to-all Token 交互。

9. 总计算量 vs 串行深度：不要混成一个指标

问题	RNN / LSTM / GRU	标准 Self-Attention
时间轴依赖	强: h_t 依赖 h_{t-1}	层内各 Token 可矩阵化并行
长程最短路径	随 Token 距离增长	单层可直接交互
标准注意力 score 成本	没有 $T \times T$ score matrix	$O(T^2)$ 个 pair
长序列内存直觉	状态递推较节省; 训练仍需保存激活	标准 Attention 对 T 很敏感
流式在线处理	天然适合一步一步更新 state	需要 KV Cache / 特殊设计

因此不能只说“Transformer 计算复杂度更好”。标准 Attention 在长序列下的 T^2 甚至可能更贵。Transformer 在大规模 LLM 中真正关键的优势，是更短的 Token 交互路径 + 更适合现代硬件的大矩阵并行 + 易于堆深扩展。

论文中如果有人只用一个 Big-O 就下结论，要继续问：比较的是 FLOPs、sequential depth、memory、wall-clock、吞吐量，还是 latency？这些不是同一个指标。

10. 动态路由: Attention 与 RNN 的另一个思维差异

RNN 把历史不断压进固定 Hidden State。虽然不同输入会产生不同状态，但信息必须通过同一个递推“瓶颈”继续向后传。

Self-Attention 则更像动态路由：对于不同 Query，可以临时决定“这一次应该从哪些 Token 读取信息，以及各读多少”。

$$\text{weight}_{\{ij\}} = \text{softmax}(\text{score}(\text{Query}_i, \text{Key}_j))$$

例如在一句话里，某个代词可能需要关注几十个 Token 以前的实体；另一个 Token 可能只需要关注相邻位置。Attention 为每个 Query 构造不同的权重分布。

这也是为什么 Attention Matrix 经常被可视化。但要注意: Attention Weight 并不自动等于因果解释或完整特征重要性。它首先是模型内部的一种信息聚合权重。

从 Agent Systems 视角，这种“按当前 Query 从历史上下文动态取信息”的思想也能帮助你理解 Agent Memory / Retrieval: 只是工程上的 Memory Retrieval 不等于 Transformer 内部 Attention。

11. 为什么 LSTM/GRU 仍然没有“结构上解决”并行问题？

LSTM 通过 Cell State 和 Gate 改善长期信息与 Gradient Flow; GRU 用更紧凑的门控管理 Hidden State。但它们仍然满足：

$$\text{state}_t = f(x_t, \text{state}_{\{t-1\}})$$

也就是说，即使长期记忆问题缓解了，时间轴的数据依赖仍然存在。因此 LSTM/GRU 的改进重点主要是：

- 让状态更新更可控；
- 让某些历史信息更容易保留；
- 让长期梯度更容易传播。

Transformer 的改动更激进：它不再要求所有历史信息必须先压进一个递推 state 才能到达后面的位置。每个 Token 的表示可以直接从一组历史 Token 表示中读取。

这就是第 50 讲与第 51 讲的关键连接：LSTM 改善递推；Transformer 改变序列交互范式。

12. 一个统一比较：MLP、CNN、RNN、Transformer 各自相信什么？

架构	核心 Inductive Bias	最自然的数据关系
MLP	特征全连接，无显式空间/时间共享结构	固定维度向量
CNN	局部连接 + 空间参数共享	图像、局部空间结构
RNN/LSTM/GRU	递推 state + 时间参数共享	顺序、流式序列
Transformer	Token 间动态 Attention 路由	大规模序列、全局交互

“Transformer 成功”并不意味着 Inductive Bias 越少越好，而是它的 Bias 与大规模数据、并行硬件和预训练目标结合得非常好。CNN 在视觉局部结构上仍然有强 Bias，RNN 在流式状态更新上仍然自然。

以后看到新架构，不要只问“性能更高吗”，还要问：它改变了什么信息路径？增加了什么 Bias？减少了什么串行依赖？代价转移到哪里？

13. 重新连接你以前学过的 GPT / LLM

现在可以把以前 LLM 课程里几个零散概念重新连起来:

LLM 概念	本讲底层解释
Q/K/V	把所有 Token 一次投影成可用于动态匹配和聚合的表示
Causal Mask	允许训练矩阵并行, 同时禁止未来信息泄漏
Multi-Head Attention	并行建立多组不同的 Token-to-Token 路由
KV Cache	解决自回归生成时历史 K/V 重复计算问题
Long Context	主要受 T^2 Attention、激活/缓存显存和实际利用能力约束
Transformer Block	Attention 负责 Token 间交互; MLP 负责每个 Token 内部特征变换

尤其要牢记: Transformer 的 Linear / MLP 仍然主要逐 Token 工作; 真正让不同 Token 直接交换信息的是 Attention。这个区分你在第 6 讲和后面的 FFN 中已经见过。

14. 与 Agent Systems 的关系：不要把“序列模型优势”直接等同于“Agent 系统优势”

Agent 通常使用 Transformer/LLM 作为认知核心，因此本讲解释了其底座为什么能高效处理长文本上下文。但 Agent System 还多了新的系统级状态：

- Tool call 与真实环境反馈；
- 长期 Memory / Retrieval；
- 多 Agent 通信；
- 任务规划与中间决策；
- 外部观察必须等前一步行动完成。

这些系统级依赖经常重新引入串行链。例如：

思考 → 调工具 → 等结果 → 观察 → 再思考

因此，Transformer 在 Token 计算层面的并行性，并不会自动让整个 Agent workflow 都可并行。你未来研究 Agent Systems 时，需要区分：

- 模型内部 Token-level dependency；
- 模型调用级 dependency；
- 工具/环境级 dependency；
- 多 Agent 协作 dependency。

这是从“深度学习组件”走向“AI Systems Science”时非常重要的一层抽象。

15. 科研阅读：看到 “RNN vs Transformer” 应该检查什么？

以后论文声称某架构更适合长序列或更高效时，可以按下面顺序检查：

检查项	具体追问
任务	长程依赖真的重要吗？还是局部模式已经足够？
序列长度	T 有多大？训练与推理是否相同？
公平性	参数量、训练 tokens、steps、数据、精度是否控制？
计算指标	比较 FLOPs、吞吐、latency、显存，还是理论复杂度？
硬件	GPU/TPU 是否更偏好大矩阵运算？
生成	训练并行优势是否在 autoregressive decoding 中仍成立？
长上下文	只是能输入更长，还是任务上真正利用了更远信息？
统计	多 seed、误差条、稳定性、失败案例是否报告？

特别注意：“最大上下文窗口更长”不等于“有效长程依赖能力更强”。输入容量、检索利用率、注意力分配、位置编码和任务实际收益是不同问题。

16. 常见误区与本讲最终闭环

误区 1: Transformer 总计算量一定比 RNN 少。

不对。标准 Attention 有 T^2 pair 成本；优势很大一部分来自更短依赖路径和更好的矩阵并行。

误区 2: Causal Mask 意味着训练也必须逐 Token。

不对。训练时整个已知序列可以一次构造 Masked Attention Matrix。

误区 3: Transformer 生成也完全并行。

不对。GPT 自回归生成仍然逐 Token；KV Cache 只是避免重复算历史 K/V。

误区 4: Transformer 已经彻底解决长程依赖。

不对。它提供更短路径，但仍有 T^2 成本、位置/检索利用、有限上下文和训练分布等问题。

误区 5: RNN 已经完全没有研究/工程价值。

不对。流式状态、小设备、固定状态内存、特定时序任务仍可能适合递推模型；现代 State Space Model 也再次探索高效序列结构。

本讲最终心智模型：

RNN：把历史递推压进 state；Self-Attention：让 Token 直接动态读取其他 Token。Transformer 在大规模预训练时代胜出的关键，不是单一公式，而是“短交互路径 + 大矩阵并行 + 可扩展堆叠”的组合；代价则转移为 T^2 Attention、显存和自回归解码串行。

下一讲：第52讲《Embedding 的深度学习本质：Token ID 怎样变成可学习向量》——从这里开始正式进入 Transformer 底层重建阶段。